

langchain家族框架对比

langchain-core:

核心抽象层/基础库，定义了最基础的接口、类型系统和抽象类
砖块和水泥

langgraph:

运行时/编排与执行引擎, 基于langchain-core, 构建相对固定长流程的状态图
施工队和脚手架

langchain:

框架应用层, 基于langchain-core和langgraph, 提供了构建智能体的简易函数
毛坯房框架

deepagents:

多智能体套件(agent harness), 基于langchain和langgraph, 提供简洁易用的规划,
精装全配豪宅

如何选择

LangGraph: 明确的多步骤长流程处理需要 (如: RAG项目)

LangChain: 简单, 快速, 没有复杂的长流程处理需求

DeepAgents: 比较复杂, 需要进行多个方向处理和分析 (如: 网络搜索, 数据库为查询, RAG)

DeepAgents核心能力

1. 智能规划与任务分解: 利用write_todos等工具将复杂任务拆解成小任务执行, 实时跟踪
2. 高效上下文管理: 内置文件系统工具来管理上下文数据, 可以及时保存到不同类型的存储中
3. 子代理生成机制: 将子代理封装成task工具, 将不同任务、复杂任务交给合适的子agent处理
4. 长期记忆能力: 多种存储方式实现长期记忆 (文件, 内存KV存储, 组合式存储)

DeepAgents基本使用

下载包

配置BASE_URL和API_KEY

定义需要的工具

初始化模型

创建深度智能体

调用智能体, 读取结果数据

Message的分类

SystemMessage - 系统消息

设定 AI 助手的角色和行为准则

例如: "你是一个专业的Python编程助手"

HumanMessage - 用户消息

代表用户的输入或问题
例如: "你好, 介绍一下你自己"
AIMessage - AI 消息
代表模型处理的回复, 可能是最终结果, 也可能是准备调用工具或者子智能体
存储模型处理的结果
ToolMessage - 工具消息
代表工具处理的返回
存储工具处理的结果

流式输出

stream()
基于 LangGraph 提供流式输出的支持, 可以实时跟踪进展、Token的使用数量和工具调用
可以判断是工具将要调用还是工具调用返回, 还是模型调用返回?
输出不同阶段的相关数据

异步执行

agent.ainvoke(): 异步非流式输出
agent.astream(): 异步流式输出 => 其返回的是AsyncIterator(异步迭代器), 必须通
在高并发服务器(如:FastAPI)中使用astream()实现流式并行执行, 提高处理速度

子智能体

- 什么时候使用SubAgent?
 - 多步骤任务会让主代理的上下文变得多而杂乱
 - 需要不同模型能力的任务(多模态)
 - 需要不同的“专业技能 / 专属工具”
- 什么时候不应使用SubAgent
 - 任务简单, 一步就能干完
 - 需要中间信息连贯, 不能拆
 - 当运营费用超过收益时
- 子agent配置的2种形式?
 - 字典
 - CompiledSubAgent
 - 字典形式的子agent
 - name / description / system_prompt / tools / model
- CompiledSubAgent形式的子agent
 - 包装langchain的agent
 - 包装langgraph的stateGraph
- 子智能体格式化输出
 - 给予智能体配置response_format, 并指定包含的字段和类型
- 子智能体可以嵌套, 但不要超过2层子智能体

人工审批

- 什么是人工审批？
对于一些高危、敏感工具操作(比如：删表删文件)，需要在执行工具进行是否能执行的审核，
- 编码实现人工审批流程？
 1. 创建agent时，配置不现工具支持审批操作 (approve,reject,edit) 和配置checkpointer
 2. 第1次执行，配置thread_id(会话id)，这次不会执行工作，而是在工具执行前中断
 3. 根据第1次执行返回的结果中包含的用户数据和中断配置信息，进行逻辑判断处理，来
 4. 第2次执行，携带审批结果和第1次执行一样的thread_id配置，这样它就会读取第1次
- 配置checkpointer的作用？
在第1调用中断停止前，自动保存状态数据，给第2次执行使用，用于第2次从此处继续向后执
- 如何让agent多次执行使用同一个会话？
执行时配置的thread_id值是一样的，只有这样才能让第2次执行读取到第1次执行中断时保

后端存储

- 作用：实现跨会话的数据共享 =》长期记忆
- 常用形式：
 - 本地文件存储： FileSystemBackend
 - 内存存储： StoreBackEnd + InMemoryStore
 - 混合存储： CompositeBackEnd，默认用文件存储，特定路由用内存存储
- 区别长期记忆与短期记忆
 - 短期记忆：保存会话的中间状态，实现同会话的多次调用间的state共享 =》人工审核
 - 长期记忆：使用本地文件或store内存保存一次会话的结果数据，其它会话可以读取保

文件权限控制

- 默认FileSystemBackend指定工作目录下都可以进行文件读写操作
- 可以通过permissions配置不同的FilesystemPermission来规定不同目录下有不同的文

技能

1. 将需要的skill添加到项目的某个目录下（不是固定，用skills比较多）
2. 配置skill位置： FileSystemBackend指定基础目录， skills来指定后面的子目录
3. skill的渐近式披露：
开始只加载所有skill的MD文件的元数据(name和desription)，用于后面匹配
当要处理用户请求时，根据加载的元数据匹配对应的skill，此时才去加载skill的全部MD

测试题

1. langchain,langgraph与deepagents的区别
2. 如何选择langchain,langgraph与deepagents?
3. Message的分类和各自的特点
4. 深度智能体执行的4个方法和区别

5. 配置子智能体的几种方式和选择
6. 说说你对人工审批的理解和编码流程
7. 说说checkpointer在人工审批中的作用
8. 说说Backends是什么？
9. 区别一下短期记忆与长期记忆？
10. deepagents中的permission是什么？
11. deepagents中skill的渐进式暴露是什么？